Table 3: Performance of Mixer and other models from the literature across various model and pre-training dataset scales. "Avg. 5" denotes the average performance across five downstream tasks. Mixer and ViT models are averaged over three fine-tuning runs, standard deviations are smaller than 0.15. (‡) Extrapolated from the numbers reported for the same models pre-trained on JFT-300M without extra regularization. (☎) Numbers provided by authors of Dosovitskiy et al. [14] through personal communication. Rows are sorted by throughput.

	Image size	Pre-Train Epochs	ImNet top-1	ReaL top-1	Avg. 5 top-1	Throughput (img/sec/core)	TPUv3 core-days
Pre-trained on ImageNet (with extra regularization)							
● Mixer-B/16	224	300	76.44	82.36	88.33	1384	0.01k(‡)
● ViT-B/16 (☎)	224	300	79.67	84.97	90.79	861	0.02k(‡)
● Mixer-L/16	224	300	71.76	77.08	87.25	419	0.04k(‡)
● ViT-L/16 (☎)	224	300	76.11	80.93	89.66	280	0.05k(‡)
Pre-trained on ImageNet-21k (with extra regularization)							
● Mixer-B/16	224	300	80.64	85.80	92.50	1384	0.15k(‡)
● ViT-B/16 (☎)	224	300	84.59	88.93	94.16	861	0.18k(‡)
● Mixer-L/16	224	300	82.89	87.54	93.63	419	0.41k(‡)
● ViT-L/16 (☎)	224	300	84.46	88.35	94.49	280	0.55k(‡)
● Mixer-L/16	448	300	83.91	87.75	93.86	105	0.41k(‡)
Pre-trained on JFT-300M							
● Mixer-S/32	224	5	68.70	75.83	87.13	11489	0.01k
● Mixer-B/32	224	7	75.53	81.94	90.99	4208	0.05k
● Mixer-S/16	224	5	73.83	80.60	89.50	3994	0.03k
● BiT-R50x1	224	7	73.69	81.92	—	2159	0.08k
● Mixer-B/16	224	7	80.00	85.56	92.60	1384	0.08k
● Mixer-L/32	224	7	80.67	85.62	93.24	1314	0.12k
● BiT-R152x1	224	7	79.12	86.12	—	932	0.14k
● BiT-R50x2	224	7	78.92	86.06	—	890	0.14k
● BiT-R152x2	224	14	83.34	88.90	—	356	0.58k
● Mixer-L/16	224	7	84.05	88.14	94.51	419	0.23k
● Mixer-L/16	224	14	84.82	88.48	94.77	419	0.45k
● ViT-L/16	224	14	85.63	89.16	95.21	280	0.65k
● Mixer-H/14	224	14	86.32	89.14	95.49	194	1.01k
● BiT-R200x3	224	14	84.73	89.58	—	141	1.78k
● Mixer-L/16	448	14	86.78	89.72	95.13	105	0.45k
● ViT-H/14	224	14	86.65	89.56	95.57	87	2.30k
● ViT-L/16 [14]	512	14	87.76	90.54	95.63	32	0.65k

While not strictly comparable, this allows us to further extrapolate the effect of scale. We use the linear 5-shot top-1 accuracy on ImageNet as a proxy for transfer quality. For every pre-training run we perform early stopping based on the best upstream validation performance. Results are reported in Figure 2 (right), where we also include ViT-B/32, ViT-L/32, ViT-L/16, and BiT-R152x2 models.

When pre-trained on the smallest subset of JFT-300M, all Mixer models strongly overfit. BiT models also overfit, but to a lesser extent, possibly due to the strong inductive biases associated with the convolutions. As the dataset increases, the performance of both Mixer-L/32 and Mixer-L/16 grows faster than BiT; Mixer-L/16 keeps improving, while the BiT model plateaus.

The same conclusions hold for ViT, consistent with Dosovitskiy et al. [14]. However, the relative improvement of larger Mixer models are even more pronounced. The performance gap between Mixer-L/16 and ViT-L/16 shrinks with data scale. It appears that Mixer benefits from the growing dataset size even more than ViT. One could speculate and explain it again with the difference in inductive biases: self-attention layers in ViT lead to certain properties of the learned functions that are *less compatible* with the true underlying distribution than those discovered with Mixer architecture.

3.4 Invariance to input permutations

In this section, we study the difference between inductive biases of Mixer and CNN architectures. Specifically, we train Mixer-B/16 and ResNet50x1 models on JFT-300M following the pre-training

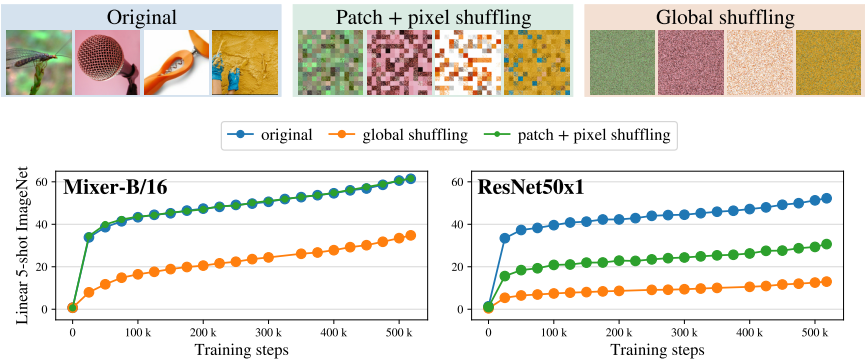

Figure 4: **Top:** Input examples from ImageNet before permuting the contents (left); after shuffling the 16×16 patches and pixels within the patches (center); after shuffling pixels globally (right). **Bottom:** Mixer-B/16 (left) and ResNet50x1 (right) trained with three corresponding input pipelines.

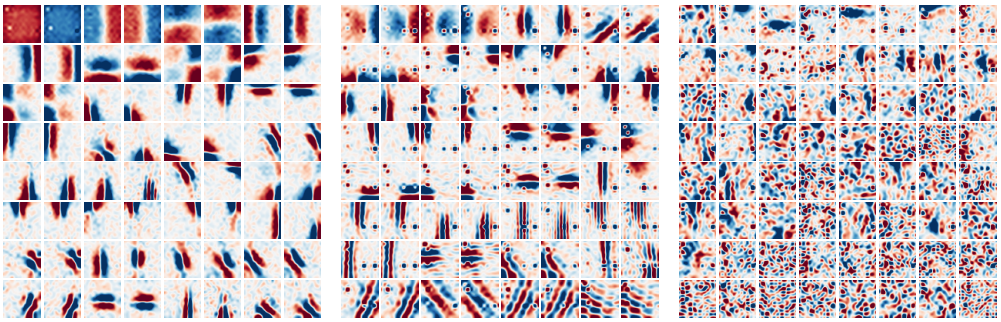

Figure 5: Hidden units in the first (**left**), second (**center**), and third (**right**) token-mixing MLPs of a Mixer-B/16 model trained on JFT-300M. Each unit has 196 weights, one for each of the 14×14 incoming patches. We pair the units to highlight the emergence of kernels of opposing phase. Pairs are sorted by filter frequency. In contrast to the kernels of convolutional filters, where each weight corresponds to one pixel in the input image, one weight in any plot from the left column corresponds to a particular 16×16 patch of the input image. Complete plots in Supplementary D.

setup described in Section 3 and using one of two different input transformations: (1) Shuffle the order of 16×16 patches and permute pixels within each patch with a shared permutation; (2) Permute the pixels globally in the entire image. Same permutation is used across all images. We report the linear 5-shot top-1 accuracy of the trained models on ImageNet in Figure 4 (bottom). Some original images along with their two transformed versions appear in Figure 4 (top). As could be expected, Mixer is invariant to the order of patches and pixels within the patches (the blue and green curves match perfectly). On the other hand, ResNet's strong inductive bias relies on a particular order of pixels within an image and its performance drops significantly when the patches are permuted. Remarkably, when globally permuting the pixels, Mixer's performance drops much less ($\sim$45% drop) compared to the ResNet ($\sim$75% drop).

3.5 Visualization

It is commonly observed that the first layers of CNNs tend to learn Gabor-like detectors that act on pixels in local regions of the image. In contrast, Mixer allows for global information exchange in the token-mixing MLPs, which begs the question whether it processes information in a similar fashion. Figure 5 shows hidden units of the first three token-mixing MLPs of Mixer trained on JFT-300M. Recall that the token-mixing MLPs allow global communication between different spatial locations. Some of the learned features operate on the entire image, while others operate on smaller regions. Deeper layers appear to have no clearly identifiable structure. Similar to CNNs, we observe many pairs of feature detectors with opposite phases [39]. The structure of learned units depends on the hyperparameters. Plots for the first embedding layer appear in Figure 7 of Supplementary D.

4 Related work

MLP-Mixer is a new architecture for computer vision that differs from previous successful architectures because it uses neither convolutional nor self-attention layers. Nevertheless, the design choices can be traced back to ideas from the literature on CNNs [24, 25] and Transformers [50].

CNNs have been the de-facto standard in computer vision since the AlexNet model [24] surpassed prevailing approaches based on hand-crafted image features [35]. Many works focused on improving the design of CNNs. Simonyan and Zisserman [41] demonstrated that one can train state-of-the-art models using only convolutions with small 3×3 kernels. He et al. [15] introduced skip-connections together with the batch normalization [20], which enabled training of very deep neural networks and further improved performance. A prominent line of research has investigated the benefits of using sparse convolutions, such as grouped [57] or depth-wise [9, 17] variants. In a similar spirit to our token-mixing MLPs, Wu et al. [55] share parameters in the depth-wise convolutions for natural language processing. Hu et al. [18] and Wang et al. [53] propose to augment convolutional networks with non-local operations to partially alleviate the constraint of local processing from CNNs. Mixer takes the idea of using convolutions with small kernels to the extreme: by reducing the kernel size to 1×1 it turns convolutions into standard dense matrix multiplications applied independently to each spatial location (channel-mixing MLPs). This alone does not allow aggregation of spatial information and to compensate we apply dense matrix multiplications that are applied to every feature across all spatial locations (token-mixing MLPs). In Mixer, matrix multiplications are applied row-wise or column-wise on the "patches$\times$features" input table, which is also closely related to the work on sparse convolutions. Mixer uses skip-connections [15] and normalization layers [2, 20].

In computer vision, self-attention based Transformer architectures were initially applied for generative modeling [8, 33]. Their value for image recognition was demonstrated later, albeit in combination with a convolution-like locality bias [37], or on low-resolution images [10]. Dosovitskiy et al. [14] introduced ViT, a pure transformer model that has fewer locality biases, but scales well to large data. ViT achieves state-of-the-art performance on popular vision benchmarks while retaining the robustness of CNNs [6]. Touvron et al. [49] trained ViT effectively on smaller datasets using extensive regularization. Mixer borrows design choices from recent transformer-based architectures. The design of Mixer's MLP-blocks originates in Vaswani et al. [50]. Converting images to a sequence of patches and directly processing embeddings of these patches originates in Dosovitskiy et al. [14].

Many recent works strive to design more effective architectures for vision. Srinivas et al. [42] replace 3×3 convolutions in ResNets by self-attention layers. Ramachandran et al. [37], Tay et al. [47], Li et al. [26], and Bello [3] design networks with new attention-like mechanisms. Mixer can be seen as a step in an orthogonal direction, without reliance on locality bias and attention mechanisms.

The work of Lin et al. [27] is closely related. It attains reasonable performance on CIFAR-10 using fully connected networks, heavy data augmentation, and pre-training with an auto-encoder. Neyshabur [30] devises custom regularization and optimization algorithms and trains a fully-connected network, attaining impressive performance on small-scale tasks. Instead we rely on token and channel-mixing MLPs, use standard regularization and optimization techniques, and scale to large data effectively.

Traditionally, networks evaluated on ImageNet [13] are trained from random initialization using Inception-style pre-processing [46]. For smaller datasets, transfer of ImageNet models is popular. However, modern state-of-the-art models typically use either weights pre-trained on larger datasets, or more recent data-augmentation and training strategies. For example, Dosovitskiy et al. [14], Kolesnikov et al. [22], Mahajan et al. [29], Pham et al. [34], Xie et al. [56] all advance state-of-the-art in image classification using large-scale pre-training. Examples of improvements due to augmentation or regularization changes include Cubuk et al. [11], who attain excellent classification performance with learned data augmentation, and Bello et al. [4], who show that canonical ResNets are still near state-of-the-art, if one uses recent training and augmentation strategies.

5 Conclusions

We describe a very simple architecture for vision. Our experiments demonstrate that it is as good as existing state-of-the-art methods in terms of the trade-off between accuracy and computational resources required for training and inference. We believe these results open many questions. On the practical side, it may be useful to study the features learned by the model and identify the main